

Experiment 03 | A* Search

[Informed Search](#) · [Heuristic Pathfinding](#) · [docs/astar.md](#) · [src/astar.py](#)

Aim

To implement the A* Search algorithm in Python and use it to find the lowest-cost path between a start node and a goal node in a weighted graph, guided by a heuristic function.

Theory

A* Search is an informed search algorithm that combines the actual cost from the start node, $g(n)$, with a heuristic estimate of the remaining cost to the goal, $h(n)$, to compute $f(n) = g(n) + h(n)$. By always expanding the node with the lowest $f(n)$, A* balances optimality and efficiency. When the heuristic is admissible (never overestimates the true cost), A* is guaranteed to return the optimal path.

Algorithm

- Add the start node to a priority queue with $f(\text{start}) = h(\text{start})$.
- Repeatedly remove the node with the lowest $f(n)$ value from the queue.
- If it is the goal, reconstruct and return the path.
- Otherwise, compute tentative g-scores for its neighbors.
- Update and re-queue any neighbor for which a cheaper path was found.
- Repeat until the goal is reached or the queue is empty.

Python Implementation

```
import heapq

def astar(graph, start, goal, heuristic):
    open_set = [(heuristic[start], start)]
    came_from = {}
    g_score = {node: float("inf") for node in graph}
    g_score[start] = 0
    f_score = {node: float("inf") for node in graph}
    f_score[start] = heuristic[start]

    while open_set:
        _, current = heapq.heappop(open_set)

        if current == goal:
            return reconstruct_path(came_from, current), g_score[current]

        for neighbor, weight in graph.get(current, []):
            tentative_g = g_score[current] + weight
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score[neighbor] = tentative_g + heuristic.get(neighbor, 0)
                heapq.heappush(open_set, (f_score[neighbor], neighbor))

    return None, float("inf")
```

Sample Input

```
graph = {
    "A": [("B", 1), ("C", 2)],
    "B": [("A", 1), ("D", 2)],
    "C": [("A", 2), ("E", 1)],
    "D": [("B", 2), ("E", 1)],
    "E": [("C", 1), ("D", 1)],
}
heuristic = {"A": 4, "B": 3, "C": 1, "D": 1, "E": 0}
start_node = "A"
goal_node = "E"
```

Output

```
Running A* Search from 'A' to 'E'...

Path found:
A -> C -> E
Total cost: 3
```

Complexity Analysis

Metric	Complexity	Notes
Time	$O(E)$	Worst case; strongly depends on heuristic quality
Space	$O(V)$	Stores g-scores, f-scores, and open/closed sets

Applications

- Pathfinding in video games and robotics navigation
- Route planning in maps and GPS systems
- Puzzle solving, e.g. the 8-puzzle and 15-puzzle
- Network routing optimization